CsEUE203-OOP(JAVA)

PROBLEM:- 1(1)	1. Vending machine (VendingMachine.java, main in this file) (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far. (d) Stop the loop once the total reaches 15 or more. (e) Print the change to return (total − 15). Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

CODE :-

```
import java.util.*;


enum Coin{
   ONE,
   TWO,
   FIVE,
   TEN
}


public class VendingMachine {    public
static void main(String args[]){        int
total = 15;
      int sum = 0;
      Scanner sc = new Scanner(System.in);


      while(sum < total){
         System.out.println("ONE | TWO | FIVE | TEN ");
         String c = sc.nextLine();


         Coin myC = Coin.valueOf(c);


         switch (myC) {
case ONE:
sum += 1;
break;


            case TWO:
sum += 2;
break;


            case FIVE:
sum += 5;
break;


            case TEN:
sum += 10;
break;
```

```
            default:
break;
        }
    }


    int last = sum - total;
System.out.print(last);


    sc.close();
  }
}
```

OUTPUT :-

```
PS D:\JAVA_OOP> cd "d:\JAVA_OOP"
PS D:\JAVA_OOP> cd "d:\JAVA_OOP\" ; if ($?) { javac VendingMachine.java } ; if ($?) { java VendingMachine }
ONE | TWO | FIVE | TEN
ONE
ONE | TWO | FIVE | TEN
TWO
ONE | TWO | FIVE | TEN
FIVE
ONE | TWO | FIVE | TEN
TEN
3
PS D:\JAVA_OOP>
```

Problame_1.2

Toll booth (TollBooth.java, main in this file)

(a) Define a record Vehicle(String number, String type).

(b) In main, keep a running toll total and three counters (bike, car, truck).

(c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter.

(d) After the loop, print the total toll and the type with the highest count.

Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.

CODE:-

```
import java.util.*;


public class TollBooth {


   public static int Vehicle(int number, String type) {
int n = 0;
      if (type.equals("B")) {
n = number * 20;
      } else if (type.equals("C")) {
n = number * 50;
      } else if (type.equals("T")) {
n = number * 150;
      }
      return n;
   }


   public static void main(String args[]) {
Scanner sc = new Scanner(System.in);
int sum = 0;


      while (true) {
         System.out.print("Bike : B | Car : C | Truck : T | Exit : Q ");
```

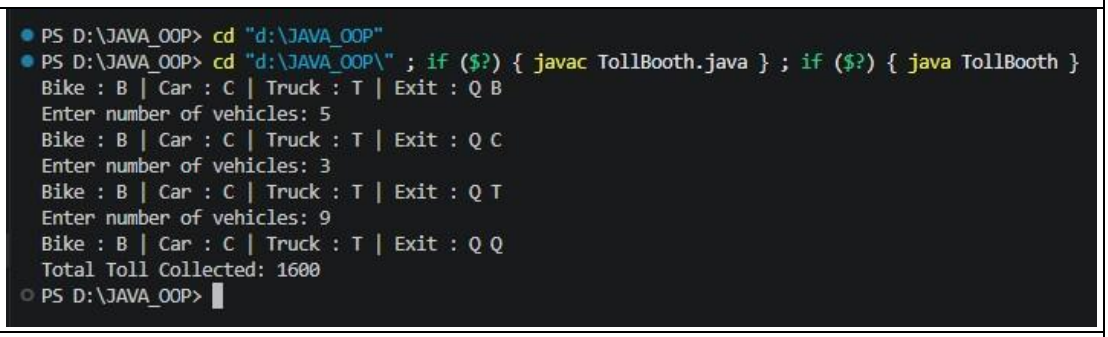
```
         String veh = sc.nextLine();


         if (veh.equals("Q")) break;


         System.out.print("Enter number of vehicles: ");
int num = sc.nextInt();
         sc.nextLine();


         sum += Vehicle(num, veh);
      }


      System.out.println("Total Toll Collected: " + sum);
sc.close();
   }
}
```

OUTPUT:-	
Problame_1.3	(For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java) (a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases). (c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score. (d) After 5 rounds, print the overall winner. Expected output: each round prints both moves and the round winner; ends with a "You win 3–2" summary.

CODE:-

```
import java.util.*;
class  RPSLS
{
   enum         Move{ROCK,PAPER,SCISSORS,LIZARD,SPOCK}
static int winner(Move a, Move b)
   {
      if(a==b)
return 0;
      switch (a)
      {
         case ROCK:
            if(b==Move.SCISSORS ||
b==Move.LIZARD)              return 1;              else
return -1;
```

```
        case PAPER:
          if(b==Move.ROCK||b==Move.SPOCK)          return 1;          else
return -1;          case SCISSORS:
          if(b==Move.PAPER||b==Move.LIZARD)          return 1;          else
return -1;          case LIZARD:
          if(b==Move.PAPER||b==Move.SPOCK)          return 1;          else
return -1;          case SPOCK:
          if(b==Move.ROCK||b==Move.SCISSORS)          return 1;          else
          return -1;
    }
    return 0;
  }


  public static void main(String arg[])
  {
    Scanner sc=new Scanner(System.in);
    Random r=new Random();       Move moves[]=Move.values();       int
playerScore=0,compScore=0;
    for(int i=1;i<=5;i++)
    {
      System.out.println("Enter your move:");
        Move player=Move.valueOf(sc.next().toUpperCase());
        Move computer=moves[r.nextInt(moves.length)];
        System.out.println("Your move:"+player);
        System.out.println("Computer  move:"+computer);                                    int
result=winner(player,computer);          if(result==1)
        {
          System.out.println("You won!");          playerScore++;
        }
        else if(result==-1)
        {
          System.out.println("Computer won!");          compScore++;
        }          else
        System.out.println("It's a tie!");
    }
    System.out.println("\nFinal Score:");
```

```
        System.out.println("Your           Score:"+playerScore);
System.out.println("Computer's       Score:"+compScore);
if(playerScore>compScore)
        System.out.println("You are the final winner!");
else if(playerScore<compScore)
        System.out.println("Computer is the final winner!");
else
        System.out.println("It's a Tie!");
    }
}
```

OUTPUT

```
PS D:\JAVA_OOP> cd "d:\JAVA_OOP\" ; if ($?) { javac RPSLS.java } ; if ($?) { java RPSLS }
Your move:SCISSORS
Computer move:LIZARD
You won!
Enter your move:
LIZARD
Your move:LIZARD
Computer move:LIZARD
It's a tie!
Enter your move:
SPOCK
Your move:SPOCK
Computer move:ROCK
You won!
Enter your move:
```